



**UNIVERSITÉ
DE LORRAINE**



Rapport
de stage

Méthodes efficaces d'apprentissage pour l'adaptation des modèles de langue

Tutrice de stage :
Mme Estelle Zheng

Co-tuteur :
M. Christophe Cerisara

Référente universitaire :
Mme Hee-Soo Choi

Présenté par :
Choplin Alexandre

L3 MIASHS - Option TAL
Année Académique 2024/2025

Remerciements

Je tiens à remercier tout particulièrement Estelle Zheng, ma tutrice de stage, pour son organisation efficace et son accompagnement tout au long de cette expérience.

Je remercie également Hee-Soo Choi, qui m'a aidé à trouver ce stage et m'a supervisé pendant sa durée, Christophe Cerisara ainsi que toute l'équipe Synalp pour leur esprit d'équipe et pour les échanges enrichissants qui ont rendu ce stage encore plus agréable.

Enfin, merci à toutes les personnes que j'ai croisées durant ce stage et qui ont contribué à rendre cette expérience formatrice et motivante.

Table des matières

1	Introduction	4
1.1	Contexte général	4
1.2	Problématique	4
1.3	Objectif	4
2	Écosystème de la recherche en IA	5
2.1	L'intelligence artificielle entre secteur privé et recherche publique	5
2.2	La recherche publique en Europe et en France	5
3	Présentation du laboratoire d'accueil	7
3.1	Le LORIA	7
3.2	L'équipe SyNaLP	7
3.3	Encadrement et conditions de travail	7
4	Le projet de stage	8
4.1	Objectifs de la mission	8
4.2	Approche adoptée	8
4.3	Contraintes et moyens techniques	8
4.4	Organisation du travail	8
5	Mes premiers pas	10
5.1	Les modèles fondation	10
5.2	Le choix du modèle	10
5.3	Le jeu de données	11
5.4	La méthode d'apprentissage	11
5.5	Les entraînements et évaluations	12
6	Pousser la réflexion	14
6.1	Des résultats inattendus	14
6.2	Entraînement sur 360 000 exemples MNLI	14
6.3	Gestion de l'infrastructure et des ressources	14
6.4	Conclusion de l'expérience	15
7	Une nouvelle expérience	16
7.1	Et en maths ?	16
7.2	Entraîner un modèle de vision pour comprendre la boucle d'entraînement	16
7.3	Application des nouvelles connaissances	17
7.4	Analyse de la longueur des raisonnements	17
7.5	Conclusion	18

8	Implémentation d'une méthode récente d'adaptation de modèle	19
8.1	Pourquoi coder cette méthode moi-même?	19
8.2	Comprendre le fonctionnement du Ladder Side Tuning	19
8.3	Sélection des parties essentielles du modèle	20
8.4	Résultats obtenus et limites rencontrées	21
8.5	Conclusion	21
9	Conclusion	22
9.1	Bilan des acquis	22
9.2	Réflexions personnelles sur le stage	22
9.3	Ouvertures possibles et continuité du travail	23
10	Annexes	24

Chapitre 1

Introduction

1.1 Contexte général

L'intelligence artificielle (IA) connaît depuis quelques années un important développement, notamment dans le domaine du traitement automatique des langues (TAL). Cette discipline vise à permettre aux ordinateurs de représenter, analyser et générer du langage naturel. C'est de ce domaine de recherche qu'ont émergé les LLMs [2] (Grands Modèles de Langue en français), modèles capables de reconnaître et de traiter du texte, et bien d'autres tâches. Les modèles les plus populaires comme ChatGPT ou Mistral font parler d'eux régulièrement dans les journaux et sont utilisés par de plus en plus de personnes.

1.2 Problématique

Bien que très puissants, ces modèles sont complexes à adapter à des tâches spécifiques, dû à leurs entraînements qui visent à les rendre le plus versatile possible. Pour cela, on utilise une technique appelée *finetuning* [9], qui consiste à réentraîner un modèle déjà entraîné sur des données plus ciblées. Cela permet d'éviter de devoir entraîner un nouveau modèle de zéro, entraînement qui coûterait plusieurs centaines de milliers d'euros en serveur et matériel informatique, voire des dizaines de millions pour les plus gros modèles. Mais même lorsque l'on se restreint à un "*finetuning*" d'un modèle existant, l'opération nécessite aussi beaucoup de mémoire et de ressources informatiques, ce qui en limite l'accessibilité, notamment pour les structures disposant de peu de moyens.

1.3 Objectif

L'objectif général du stage était de contribuer au développement d'une nouvelle méthode qui permet de réentraîner un modèle avec beaucoup moins de ressources matérielles. Ce travail s'est caractérisé par de l'écriture de code en Python, et des lectures d'articles scientifiques.

Chapitre 2

Écosystème de la recherche en IA

2.1 L'intelligence artificielle entre secteur privé et recherche publique

Les modèles d'intelligence artificielle tels que les LLM sont de plus en plus utilisés par les GAFAM, géants de la tech qui les incluent dans leurs services pour leurs utilisateurs. Les recherches Bing sont peu à peu délaissées et remplacées par des requêtes à ChatGPT. En effet, les modèles de langage de grande taille (LLM) sont particulièrement faciles à utiliser et contribuent à simplifier de nombreuses tâches pour les utilisateurs, telles que la rédaction de courriels, la recherche et la synthèse d'informations, entre autres. Par conséquent, ces entreprises les intègrent directement dans les smartphones : Gemini, qui est l'assistant des téléphones Google Pixel, est désormais accessible en maintenant appuyé le bouton démarrer du smartphone. Meta AI est lui directement intégré sur WhatsApp, Instagram et Facebook pour faciliter les recherches ou lui poser des questions.

Chacune de ces entreprises a le budget, les ressources et les développeurs pour créer des LLM de la plus haute qualité, qualité dont elles gardent le secret de fabrication, ou au moins le droit d'utilisation commerciale. Ce qui est tout à fait raisonnable étant donné que chaque "modèle fondation" coûte cher.

Ces entreprises se positionnent alors comme pionnières du développement de l'intelligence artificielle, mettant en avant leur avance sur la concurrence dans un domaine qu'elles présentent, lors de leurs conférences de presse, comme une nouvelle révolution industrielle. Ces découvertes technologiques permettent de concevoir des outils rentables qui génèrent des investissements. Elles encouragent donc tout le secteur de l'informatique à s'intéresser au sujet, pour créer des applications utilisant les capacités des modèles toujours plus puissants, ainsi que les institutions publiques des plus grosses puissances mondiales comme la Chine, les États-Unis, ou la France.

2.2 La recherche publique en Europe et en France

Face à l'avance prise par les grandes entreprises américaines dans le domaine des modèles de langage, l'Union européenne cherche à maintenir une certaine autonomie technologique [3]. Elle investit environ un milliard d'euros par an dans le développement de l'intelligence artificielle via des programmes comme Horizon Europe [6] et Digital Europe [5]. Ces financements servent à soutenir la recherche, développer des modèles, et favoriser le transfert des technologies vers l'industrie.

L'Europe souhaite rester compétitive tout en développant une IA "de confiance", plus transparente et conforme à ses valeurs, notamment en matière de respect de la vie privée ou de lutte contre les biais.

En France, ces financements se traduisent par des projets portés par des établissements

publics comme le CNRS. Ces laboratoires n'ont pas les moyens de construire des modèles aussi grands que ceux de Google ou OpenAI, mais se concentrent sur des alternatives plus légères, souvent mieux documentées, reproductibles, et ouvertes à la communauté scientifique. Cela permet de tester d'autres approches (frugalité, robustesse, interprétabilité, optimisations) et d'assurer une recherche indépendante.

Chapitre 3

Présentation du laboratoire d'accueil

3.1 Le LORIA

Le stage a été réalisé au Loria (Laboratoire Lorrain de Recherche en Informatique et ses Applications), un centre public de recherche en informatique basé à Nancy. Fondé en 1997, le LORIA est une Unité Mixte de Recherche (UMR 7503), placée sous la tutelle du CNRS, de l'Université de Lorraine, de CentraleSupélec et de l'Inria. Il regroupe plus de 500 personnes réparties dans une trentaine d'équipes, couvrant des thématiques variées comme l'intelligence artificielle, la cybersécurité, les algorithmes, ou la robotique.

3.2 L'équipe SyNaLP

Mon stage s'est déroulé au sein de l'équipe Synalp (Syntax, Semantics and Natural Language Processing), qui mène des recherches depuis 2012 dans le domaine du traitement automatique du langage naturel. L'équipe est composée de doctorants, d'ingénieurs de recherche et de chercheurs académiques du CNRS. L'équipe s'intéresse notamment à la génération et à la compréhension de texte, à l'analyse de documents multilingues, à l'apprentissage en faible ressources, ainsi qu'aux LLM. L'équipe Synalp ainsi que plusieurs autres équipes du Loria ont été regroupées récemment dans un super-groupe appelé Mosaik [10] qui a été créé en 2024 dans le prolongement des anciennes équipes orientées IA.

3.3 Encadrement et conditions de travail

J'ai été encadré par Estelle Zheng, doctorante en contrat CIFRE. Elle m'a guidé tout au long du stage, en me proposant des objectifs adaptés à mon niveau pour me faire progresser sur ma compréhension de son projet. Nous avons une réunion hebdomadaire chaque lundi après-midi, et je pouvais lui demander de l'aide en cas de blocage ou de besoin de validation.

Mon poste de travail se trouvait dans une salle partagée avec d'autres stagiaires, à proximité des membres de l'équipe Synalp. J'utilisais mon propre ordinateur portable. J'ai également participé à des réunions de bienvenue des nouveaux employés du Loria durant lesquelles on m'avait présenté les différentes salles, et la structure des bâtiments.

Chapitre 4

Le projet de stage

4.1 Objectifs de la mission

Mon stage s’inscrivait dans le cadre de la thèse de ma tutrice, financée en partie par le CNRS, thèse qui porte sur l’étude de méthodes d’adaptation des LLM. L’idée est de travailler sur des architectures plus petites et/ou spécialisées, afin de mieux comprendre leur fonctionnement et de proposer des alternatives plus légères aux grands modèles privés, dont les détails restent souvent confidentiels.

L’objectif de ma mission était de comparer différentes méthodes de *fine-tuning*, c’est à dire des techniques permettant d’adapter un modèle de langue déjà entraîné sur d’autres données à une tâche spécifique, et d’en tester une nouvelle, développée dans le cadre de cette thèse. Cette méthode vise à réduire fortement la mémoire nécessaire à l’adaptation, ce qui permet de faire tourner l’entraînement sur des machines moins puissantes et avec une consommation énergétique réduite.

4.2 Approche adoptée

Pour atteindre ces objectifs, le stage a été structuré autour de plusieurs axes : une formation aux bases du *fine-tuning* de modèles de langue avec Python, une mise en œuvre des expériences pratiques d’adaptation de modèles existants à différentes tâches, l’évaluation et comparaison des performances des approches classiques et de la méthode innovante proposée dans le cadre de la thèse.

Le travail a porté sur des modèles en libre accès, en les adaptant à des tâches de classification de texte ou de résolution de problèmes mathématiques. J’ai également implémenté le code d’une méthode décrite par une publication scientifique.

4.3 Contraintes et moyens techniques

L’adaptation de LLM demande un accès à des ressources matérielles spécifiques, notamment des cartes graphiques avec beaucoup de mémoire. Pour cela, j’ai utilisé la plateforme Grid5000, qui permet de réserver des serveurs équipés de matériel adapté à ce type de calcul intensif. Grid5000 est un réseau de serveurs dans plusieurs villes de France, dont un est à Nancy. Cette dépendance technique a parfois freiné l’avancée du projet, notamment lors des périodes de maintenance ou de saturation des serveurs.

4.4 Organisation du travail

Le projet a été mené dans un cadre relativement souple mais structuré. Je travaillais de façon autonome, en suivant une progression par étapes définie avec ma tutrice. Elle m’a transmis les objectifs à atteindre, m’a formé aux outils nécessaires, et nous faisions régulièrement le point

sur l'avancement. Le suivi s'est principalement fait via une réunion hebdomadaire les lundis après-midi, mais je pouvais aussi poser des questions ou demander de l'aide dès que nécessaire à Estelle Zheng. Le projet n'impliquait pas de gestion d'équipe ni de coordination avec d'autres stagiaires, hors la passation de mon travail à un stagiaire qui est arrivé durant la dernière semaine de mon stage.

Chapitre 5

Mes premiers pas

5.1 Les modèles fondation

Comme expliqué précédemment, entraîner un grand modèle d'intelligence artificielle à partir de zéro nécessite d'immenses ressources que seuls quelques géants de la tech peuvent mobiliser. C'est pourquoi les chercheurs et ingénieurs utilisent souvent des modèles déjà entraînés, mis à disposition en open source. On appelle ces modèles des modèles fondation, car ils servent de base solide pour développer d'autres systèmes plus spécialisés.

Dans mon stage, j'ai utilisé un de ces modèles, appelé Qwen-2.5 [12], publié en septembre 2024 par Alibaba Cloud (une filiale du groupe Alibaba). Ce modèle est libre d'utilisation pour la recherche, ce qui permet à de nombreux laboratoires de recherche de s'en servir comme point de départ pour leurs propres travaux.

Ce modèle libre d'accès est largement utilisé, et est disponible sur Huggingface, qui est une plateforme qui recense et classifie des centaines de modèles et jeux de données open source, à laquelle est associée une bibliothèque de code python qui rend l'accès et l'utilisation de ces modèles beaucoup plus simple.

Cette bibliothèque est donc un moyen très abordable pour débiter dans l'entraînement de LLM et commencer à apprendre comment fonctionne le processus d'adaptation étape par étape, processus dont je ne connaissais pas beaucoup d'éléments au début du stage. L'objectif de cette première expérience pratique était de me former à l'utilisation des bibliothèques de code Python spécifiques à l'entraînement d'IA, et de me familiariser avec l'utilisation des ressources informatiques de Grid5000.

5.2 Le choix du modèle

Parmi les différentes variantes existantes de Qwen-2.5, j'ai travaillé avec la version "Qwen-2.5-0.5B-Instruct". Cela signifie deux choses :

- 0.5B : le modèle contient 0,5 milliard de paramètres (les "connexions" internes qui permettent au modèle de fonctionner). Cela peut sembler énorme, mais c'est en réalité très petit pour un modèle de ce type — certains dépassent les 100 milliards de paramètres. En comparaison, le plus gros modèle de la gamme Qwen-2.5, Qwen-2.5-Max fait 72 milliards de paramètres.
- Instruct : ce suffixe indique que le modèle a été entraîné avec des données formulées sous forme d'instructions, pour mieux répondre à des consignes ou des questions.

Ce modèle relativement compact a l'avantage de pouvoir fonctionner sur une des cartes graphiques grand public disponible sur les serveurs de Grid5000. Cela le rend particulièrement adapté à des processus moins exigeants, mais à exécuter de manière répétée, en raison du besoin de plusieurs essais avant d'obtenir un fonctionnement optimal.

5.3 Le jeu de données

L'objectif de cette première étape de développement du projet est de me faire gagner en connaissances, et de me familiariser avec la bibliothèque Huggingface. L'expérience consiste en l'entraînement d'un modèle sur une tâche spécifique, et d'évaluer à quel point cette surcouche d'entraînement l'a rendu plus apte à la traiter.

La tâche d'entraînement spécifique sur laquelle j'ai entraîné mon premier modèle est une tâche de classification, c'est-à-dire que je demande au modèle de choisir entre plusieurs options la bonne réponse.

Pour entraîner un modèle, on utilise un jeu de données, c'est-à-dire un ensemble d'exemples qui se présentent dans le même format et qui contiennent le même type de problème que l'on souhaite apprendre au modèle.

Dans mon cas, j'ai utilisé le jeu de données MNLI (Multi-Genre Natural Language Inference), qui consiste à évaluer la relation logique entre deux phrases. Le modèle doit déterminer si la deuxième phrase (appelée "hypothèse") est une conséquence logique de la première, auquel cas le modèle doit dire "vrai", en contradiction avec la première et doit donc répondre "faux", ou simplement indépendante et doit dire "ni l'un ni l'autre".

C'est en présentant ces exercices un par un au modèle qu'il sera capable de représenter finement le contexte et les relations logiques générales, ce qui améliorera sa capacité de raisonnement.

Le jeu de données MNLI comprend plus de 360 000 exemples annotés, ce qui en fait une excellente base pour apprendre à mieux "comprendre" le langage, car la capacité de généralisation du modèle dépend du nombre d'exemples utilisés pour l'entraînement.

5.4 La méthode d'apprentissage

Pour présenter les exemples au modèle et qu'il apprenne d'eux, il faut définir comment le modèle évoluera à chaque nouvel exemple. C'est-à-dire qu'il faut programmer un processus par lequel le modèle s'appuiera sur les réponses précédentes pour prédire la réponse courante. Différentes méthodes existent et peuvent être définies facilement dans la bibliothèque Huggingface, d'où l'utilisation de cette bibliothèque pour cette expérience ainsi que les suivantes.

La méthode de base consiste en un réentraînement exhaustif de tous les neurones du modèle, c'est-à-dire que l'on va l'ajuster complètement à chaque exemple. Réentraîner tout le modèle est extrêmement coûteux et prend beaucoup de temps, et d'autres méthodes ont été développées pour adapter un modèle de manière moins coûteuse.

Ces méthodes, appelées PEFT (Parameter-Efficient Fine-Tuning), permettent de modifier uniquement une petite partie du modèle (une petite partie des paramètres), ou de définir de nouvelles couches de neurones, ce qui économise du temps et de la mémoire, sans faire de compromis sur la qualité de l'adaptation.

Dans mon cas la méthode d'entraînement LoRA [4] (Low Rank adaptation) consiste en la définition de petits modules que l'on va raccrocher au modèle de base, sans toucher à la structure principale dont les paramètres, les neurones, resteront tels quels. Cette méthode est déjà implémentée dans le code de la bibliothèque Huggingface, ce qui m'a permis de réaliser la tâche sans nécessairement en comprendre tout les tenants et aboutissants dès le début. La bibliothèque Huggingface propose d'utiliser des morceaux de code très modulaires qui s'imbriquent les uns dans les autres naturellement, ce qui m'a permis, en m'inspirant de leur documentation, d'écrire du code qui fonctionne sans en comprendre les détails dans un premier temps.

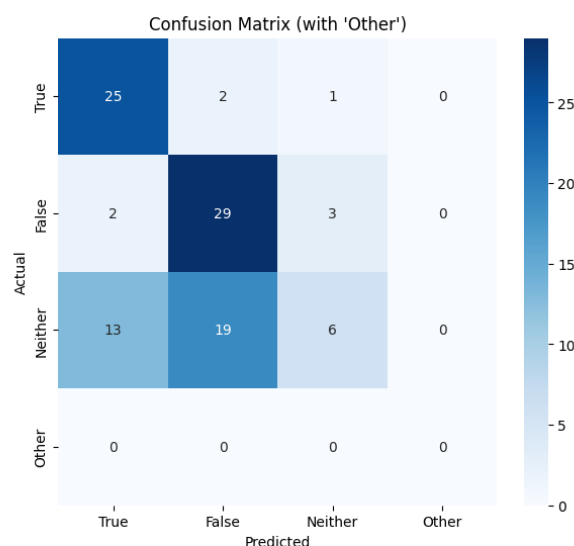
5.5 Les entraînements et évaluations

Pour évaluer mon modèle, j’ai utilisé des exemples du jeu de données MNLI que je n’avais pas montrés à Qwen-2.5-0.5-Instruct pendant l’entraînement, afin de ne pas tricher, car si le modèle avait déjà été exposé à l’exemple, il est très probable qu’il fournirait la bonne réponse, ce qui fausserait les résultats en les surestimant. En réalité, ses performances seraient alors nettement inférieures à ce qu’elles paraissent. Non entraîné, le modèle parvient à donner la bonne réponse dans 41% des cas ; il m’appartient donc d’améliorer ce score.

Mon premier entraînement a rendu le modèle aléatoire dans ses réponses. Après avoir résolu quelques erreurs je m’étais retrouvé avec un modèle d’une exactitude autour de 33%. En regardant les résultats de l’évaluation dans une matrice de confusion, j’ai pu voir que le modèle répondait presque tout le temps "Faux". Petit à petit, en résolvant les divers problèmes de mon processus d’entraînement initial, la exactitude a augmenté, et j’ai appris et mieux compris les détails du code que j’avais implémenté.

J’ai écrit mon propre processus d’évaluation du modèle, basé sur le même processus de mise en forme du jeu de données que celui de l’entraînement, ce qui a permis d’être le plus représentatif possible de ce que j’ai tenté d’instruire au modèle. De plus, avoir la main sur les détails des réponses fournies par mon modèle m’a permis de découvrir ses faiblesses durant le processus d’amélioration de mon code d’adaptation. En étant capable de pointer du doigt telle ou telle faiblesse, j’ai été capable de suspecter une partie ou une autre du processus comme étant fautive à partir des conseils d’Estelle Zheng. Cette méthode itérative de développement m’a amené à des résultats probants, en évaluant le modèle adapté sur 100 nouveaux exemples j’ai fini par atteindre 60% de exactitude d’après ma méthode d’évaluation, donc une amélioration de 19% par rapport au modèle non-entraîné.

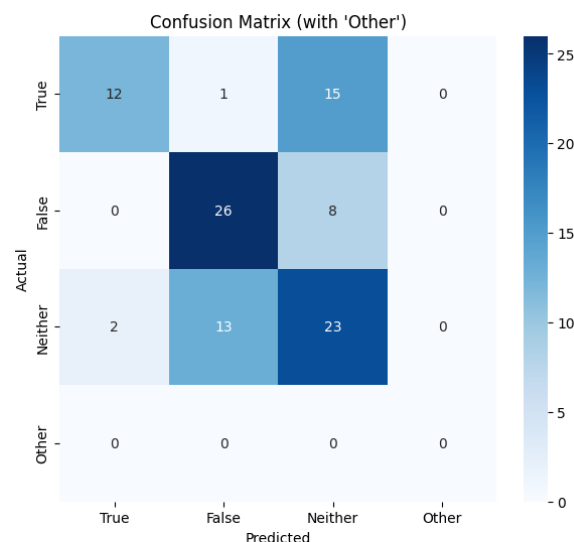
Pour mieux comprendre comment les différentes variantes de mon entraînement affectaient le comportement du modèle, j’ai comparé plusieurs matrices de confusion. Par exemple, deux entraînements successifs ont abouti à des modèles ayant des exactitudes similaires, mais des types d’erreurs sensiblement différents. Le premier modèle **Figure 5.1a** faisait principalement des erreurs en classant trop souvent les exemples « Neither » comme « True » ou « False », sans réellement différencier les cas. Le second modèle **Figure 5.1b**, entraîné avec des ajustements supplémentaires de mes hyperparamètres et du traitement des données d’entrée, montrait une meilleure capacité à éviter ces confusions systématiques, en classifiant les « True » et « False » plus correctement, malgré une légère baisse de confiance globale. Cette progression montre bien que même sans gain net de exactitude, une meilleure explicabilité des erreurs peut être atteinte grâce à un entraînement plus finement contrôlé, et en fonction du modèle que l’on veut utiliser, différentes faiblesses de classification peuvent être compensées par une utilisation intelligente d’un modèle, en prenant en compte un degré de confiance des réponses du modèle en fonction de la classe qu’il prédit.



Classification Report:

	precision	recall	f1-score	support
True	0.62	0.89	0.74	28
False	0.58	0.85	0.69	34
Neither	0.60	0.16	0.25	38
Other	0.00	0.00	0.00	0
micro avg	0.60	0.60	0.60	100
macro avg	0.45	0.48	0.42	100
weighted avg	0.60	0.60	0.54	100

(a) Le modèle fait principalement des erreurs sur la classe "Neither".



Classification Report:

	precision	recall	f1-score	support
True	0.86	0.43	0.57	28
False	0.65	0.76	0.70	34
Neither	0.50	0.61	0.55	38
Other	0.00	0.00	0.00	0
accuracy			0.61	100
macro avg	0.50	0.45	0.46	100
weighted avg	0.65	0.61	0.61	100

(b) Les erreurs sont plus réparties, avec des confusions entre "True", "Neither" et "False".

FIGURE 5.1 – Comparaison des matrices de confusion obtenues avec deux variantes d'entraînement du modèle.

Afin de valider mes résultats d'évaluation, j'ai utilisé une bibliothèque standardisée et très utilisée dans la communauté `lm-eval-harness` [8] afin de confirmer que mon évaluation n'était pas biaisée. Je me suis basé sur un fichier de code en python écrit par Estelle Zheng, que j'ai facilement pu adapter à mon cas. J'y ai défini mon modèle, et la tâche sur laquelle je voulais l'évaluer. J'ai exécuté l'évaluation sur 500 exemples, et la exactitude qui en est ressortie était de 50,8%.

La différence significative entre mon évaluation et celle de "`lm-eval-harness`" (la bibliothèque en question) peut s'expliquer par une certaine variance des résultats, ainsi qu'une méthode légèrement différente pour formater les questions, et pour récupérer les réponses. Puisque le modèle repose beaucoup sur une formulation bien précise, il n'est pas inconcevable de voir une grosse différence de exactitude entre deux méthodes marginalement différentes.

Par exemple pour expliquer au modèle ses options de réponse, j'ai trouvé que les lister de cette manière : "True or False or Neither?" apportait des meilleurs résultats à hauteur de 4% comparé à cette autre manière : "True, False or Neither?". Et si la méthode de mise en forme des exercices de classification de `lm-eval-harness` possède des subtilités que mon code n'a pas prévues, il est normal de s'attendre à des disparités entre les résultats.

Néanmoins j'ai tenté de comprendre ces différences de technique et de ré-entraîner mon modèle de manière à respecter leur méthode, mais sans succès.

Chapitre 6

Pousser la réflexion

6.1 Des résultats inattendus

Les résultats de plus de 50% de exactitude pour le fine-tuning en LoRA (avec des adaptateurs modulaires rajoutés au modèle principal) montrent que le modèle est capable de bien apprendre sur cette tâche. L'autre méthode d'adaptation de modèle qu'Estelle Zheng développe, la XLadder (Extended Ladder) qui est une variante de LST [11] (Ladder Side Tuning) atteignait un score un peu moindre entre 40 et 50%. Bien que le fait que cette méthode nécessite bien moins d'espace mémoire pour l'entraînement soit une victoire en soit (environ deux tiers plus frugal), les résultats atteints sont moindres pour la plupart des configurations testées.

Une hypothèse pour expliquer cette différence était que le modèle entraîné avec le LoRA avait beaucoup plus "désappris" (oublié) comment résoudre d'autres types de tâches. C'est une conséquence possible de l'adaptation d'un modèle, car en privilégiant une tâche spécifique, on écrase au moins un petit peu d'autres connaissances ou circuits de réflexion dans le modèle.

J'ai donc exécuté une évaluation de mon modèle entraîné sur plusieurs autres tâches qui présentent d'autres types de problèmes pour le vérifier, toujours avec la bibliothèque Python "lm-eval-harness". En comparant ces nouveaux résultats avec ceux de la variante LST, on ne voit pas de différence très notable dans leurs pertes respectives.

6.2 Entraînement sur 360 000 exemples MNLI

Pour continuer la comparaison des deux méthodes d'adaptation, j'avais pour objectif d'entraîner mon modèle au maximum de ses capacités, et Estelle Zheng le sien afin de comparer la qualité la plus haute atteignable.

Jusque là l'entraînement atteignant 50,8% de exactitude était réalisé sur une toute petite partie des données disponibles du jeu de données MNLI, les différents entraînements ont été réalisés avec 1000, puis 2000 et enfin 10 000 exemples. Mais pour voir les capacités totales d'adaptation du modèle Qwen-2.5-0.5B-Instruct à la tâche MNLI, il faut l'entraîner sur l'intégralité des données pour définir les limites inhérentes à sa structure, plus que les limites des hyperparamètres choisis pour son entraînement.

6.3 Gestion de l'infrastructure et des ressources

Néanmoins il faut avoir un environnement matériel très performant, car à cette échelle un entraînement prend plusieurs heures (5 heures estimées dans ce cas). Afin d'accélérer ce processus, il est possible de faire un entraînement sur plusieurs exemples à la fois, mais cela requiert un matériel plus puissant. Les calculs exécutés à l'intérieur des couches du modèle d'intelligence artificielle sont en grande majorité des multiplications de matrices, et cela représente plusieurs milliards d'opérations à exécuter. C'est la raison pour laquelle on exécute ces

modèles sur une carte graphique car cette architecture est composée de couches de neurones, car elles sont composées de plusieurs milliers de tout petits processeurs capables de résoudre des petites multiplications de matrices en parallèle en très grand nombre. La restriction qui arrive quand on a besoin d'une carte graphique pour exécuter du code, c'est que sa mémoire ne permet pas nécessairement de contenir tous les neurones du modèle à la fois.

Dans mon cas, pour que la carte graphique puisse retenir les changements à effectuer sur les adaptateurs du modèles pour deux, quatre ou huit exemples à la fois il faut avoir significativement plus de mémoire vive disponible. C'est pourquoi j'ai choisi de réserver une plus grosse carte graphique sur Grid5000, qui a une capacité bien plus grande.

J'ai tenté de lancer ce processus qui prend beaucoup de temps sans succès à cause de plusieurs problèmes liés à des *drivers* de carte graphiques non compatibles avec les versions de bibliothèques Python que j'avais sélectionnées, car ils ne fonctionnaient que pour une plus petite carte graphique.

Ces problèmes de bibliothèques et de *drivers* non-compatibles m'ont largement ralenti dans mon développement de toutes les expériences que j'ai réalisées durant mon stage, malgré plusieurs mesures visant à prévenir de ce genre de problèmes. Par exemple, j'ai défini des environnements séparés de python avec le gestionnaire de paquets "Conda" [1] afin de séparer les paquets utilisés en fonction de l'expérience à réaliser. Mais certains paquets python ne fonctionnent que si l'environnement CUDA (les drivers, donc les logiciels nécessaires au bon fonctionnement de la carte graphique) est assez récent.

Je n'ai donc pas eu le temps de compléter cette tâche avant qu'Estelle Zheng n'ait vraiment besoin du résultat, et qu'elle ait besoin de s'en charger elle même. Ses résultats sont donc que le modèle en adaptation avec la méthode LoRa est capable d'atteindre 86,2% de exactitude et la variante LST seulement 66%. C'est encore une fois une différence significative qui met en avant une faiblesse dans la méthode utilisée jusque là pour adapter le modèle de langue.

En analyse a posteriori de ce résultat, on se rend compte que la technique de la variante LST reposait sur un deuxième, plus petit modèle, qui était trop obsolète pour permettre d'apprendre convenablement sur cette tâche.

6.4 Conclusion de l'expérience

Qwen2.5-0.5B-Instruct a été capable d'apprendre de manière significative sur une tâche de classification. Cet entraînement a permis de définir objectivement une exactitude à atteindre pour la nouvelle méthode d'adaptation LST. Il m'a aussi été crucial pour comprendre étape par étape les différents rouages du fine-tuning (adaptation) du modèle.

Chapitre 7

Une nouvelle expérience

7.1 Et en maths ?

J'avais donc écrit du code Python pour déterminer à quel point un modèle de langue spécifique était capable de devenir meilleur à une tâche de classification, maintenant il était question de voir si la technique d'adaptation d'Estelle Zheng était capable d'obtenir de meilleurs résultats sur un autre type de tâche, la résolution d'exercices de mathématiques simples.

La méthodologie adoptée était la même que pour l'entraînement précédent, j'avais aussi utilisé Huggingface pour télécharger un modèle, et je l'avais évalué de la même manière, d'abord avec ma méthode pour la réalisation des itérations du code Python nécessaire à l'entraînement, puis à l'aide de `lm-eval-harness`.

La grosse différence entre les deux expériences était que je m'étais interdit l'utilisation de certains blocs de code de Huggingface pour me forcer à comprendre en détail leur fonctionnement en devant les réécrire par moi-même, notamment la mise en forme des jeu de données d'entraînement et leur découpage, ainsi que la boucle d'entraînement du modèle.

Comprendre le fonctionnement de boucle d'entraînement précisément représente la dernière subtilité, et la plus complexe du fine-tuning en général.

7.2 Entraîner un modèle de vision pour comprendre la boucle d'entraînement

Pour comprendre la boucle d'entraînement, Estelle Zheng a recommandé que j'utilise un autre type de intelligence artificielle, dont la structure est plus simple. En effet, la boucle d'entraînement est un processus commun à tous les neurones et est donc compréhensible de manière plus simple sur une modalité (un type) de donnée différente.

J'ai utilisé le framework PyTorch, une bibliothèque très répandue pour développer et entraîner des modèles de neurones. C'est aussi l'infrastructure sur laquelle reposent les modèles proposés par Huggingface, une plateforme de référence en intelligence artificielle qui met à disposition des modèles préentraînés, notamment via leur interface `transformers`.

Plutôt que d'utiliser directement un entraînement guidé (appelé Supervised Fine-Tuning, ou SFT) avec des outils déjà abstraits comme ceux de *transformers*, j'ai choisi d'implémenter moi-même une boucle d'entraînement complète, afin d'en comprendre chaque étape. Pour cela, j'ai entraîné un modèle CNN, modèle de vision, qui apprend à reconnaître des formes ou des objets dans une image. Je l'ai entraîné sur un type spécifique d'images à classifier : un jeu de données de chiffres manuscrits appelé MNIST, et qui atteint une exactitude de 98

La boucle d'entraînement que j'ai codée se décompose en plusieurs étapes que l'on répète à chaque passage sur les données : d'abord, le modèle reçoit un lot d'images puis il produit des prédictions. Ensuite, une fonction de perte compare ces prédictions aux bonnes réponses

pour quantifier la marge d'erreur. Cette erreur est ensuite utilisée pour calculer un ensemble de valeurs appelées gradients, qui indiquent comment ajuster les paramètres du modèle. Enfin, un optimiseur applique ces ajustements. Ce cycle est répété sur de nombreux lots, puis sur de nombreuses passes complètes des données (appelées époques), jusqu'à ce que les performances du modèle se stabilisent.

Cette performance est illustrée par la **Figure 10.2**, qui montre une matrice de confusion représentant les prédictions finales du modèle. On y voit que la majorité des chiffres sont correctement classifiés, avec très peu d'erreurs entre classes similaires (par exemple entre les chiffres 5 et 9).

Ce travail m'a permis de mieux comprendre ce que des bibliothèques de plus haut niveau masquent quand il s'agit de définir le rôle exact des différents hyperparamètres choisissables.

7.3 Application des nouvelles connaissances

Puisque que le code que j'avais écrit pour entraîner le modèle CNN est en PyTorch, et que Pytorch est la bibliothèque élémentaire de Huggingface, j'ai pu réutiliser mon code en l'adaptant pour entraîner un autre type de modèle, sur une modalité de données différente. Il m'a fallu comprendre précisément comment adapter la logique de la boucle d'entraînement à un modèle de type *transformers* (c'est-à-dire un LLM). Ce changement de modalité a entraîné un besoin de changer quelques étapes de la boucle d'entraînement qui dépendent en partie du type de modèle utilisé.

J'ai choisi d'entraîner le modèle Qwen-2.5-Math-1.5B-Instruct, une version de Qwen-2.5 optimisée pour la résolution d'exercices de mathématiques. Ce modèle a été entraîné avec des données mathématiques supplémentaires, et spécialement conçu pour produire un raisonnement intermédiaire étape par étape, selon la méthode du Chain of Thought (CoT).

L'objectif de cette expérience était double. D'une part, il s'agissait de vérifier si une adaptation fine du modèle pouvait améliorer ses performances sur le jeu de données GSM8K, une collection d'un peu moins de 9000 problèmes mathématiques de niveau scolaire, où l'on attend une réponse numérique justifiée par un raisonnement. J'ai repris la même boucle d'entraînement que celle développée auparavant, en l'adaptant au format textuel des entrées-sorties, ainsi qu'aux spécificités du modèle Qwen. Pour l'évaluation, j'ai utilisé le même outil que dans l'expérience précédente (lm-eval-harness), mais une version spécifique que les auteurs du modèle avaient incluse dans leur page GitHub d'évaluation de leur modèle.

J'ai finalement obtenu un score de 84,8% de exactitude, marginalement supérieur à l'évaluation officielle d'Alibaba Cloud qui annonçait 84,4%. Ce tout petit écart suggère que le modèle a probablement déjà vu des exemples très similaires lors de son pré-entraînement, bien que le jeu de données GSM8K ne soit pas mentionné dans leur documentation. Cela soulève des questions sur la composition réelle de leurs données. Aussi, on peut se demander si le modèle a atteint ses limites structurelles d'apprentissage qualitatif de résolution de problème ; après tout, le modèle n'est pas un très grand modèle.

7.4 Analyse de la longueur des raisonnements

Le second objectif était d'analyser la taille des raisonnements produits par le modèle, pour évaluer leur coût en termes de tokens. Cela permet de mieux comprendre l'impact du format de sortie sur l'efficacité du modèle, surtout dans des contextes où les ressources sont limitées. J'ai donc mesuré la longueur moyenne des *labels* (les raisonnements corrects fournis comme exemples pendant l'entraînement) et des prédictions générées par le modèle comme vu sur la **Figure 10.1**.

Les résultats étaient les suivants :

— Longueur moyenne des labels : 125,86 tokens

— Longueur moyenne des prédictions pendant l’entraînement : 218,45 tokens

Ces valeurs indiquent que les raisonnements générés par le modèle sont en moyenne plus longs que ceux fournis en exemple. C’est en effet un problème récurrent des LLMs qui ont tendance à être trop verbeux.

7.5 Conclusion

Cette expérience m’a permis de mettre en pratique mes connaissances acquises sur le processus d’adaptation d’un modèle de langue, en développant de manière itérative, améliorant à chaque fois certains détails des mécanismes fondamentaux utiles pour l’adaptation d’un LLM. Même si les gains de performance obtenus ne sont pas significatifs, avoir réussi à répliquer les résultats de l’équipe d’Alibaba Cloud montre que j’avais au moins compris comment une bonne partie du processus fonctionne. Ce travail m’a également montré l’importance de bien comprendre ce que font les abstractions utilisées dans les bibliothèques de haut niveau.

Chapitre 8

Implémentation d'une méthode récente d'adaptation de modèle

8.1 Pourquoi coder cette méthode moi-même ?

Dans la continuité des expériences précédentes, j'ai souhaité explorer une autre approche de l'adaptation de modèles d'IA, plus proche de celle développée par ma tutrice Estelle Zheng dans sa propre recherche. Il s'agit d'une méthode récente, appelée LST, proposée dans un article scientifique publié en 2022 [11].

Plutôt que de me contenter d'utiliser un code déjà existant, j'ai fait le choix de réimplémenter cette méthode moi-même, en repartant du papier de recherche. Cela m'a permis d'en comprendre les détails les plus fins. La méthode LST étant encore relativement récente, il n'existe à ce jour ni bibliothèque de référence, ni véritable tutoriel, ni documentation accessible en ligne. Même le code mis à disposition par les auteurs du papier s'est avéré difficile à exploiter car il est peu structuré, peu documenté, et orienté vers des modèles différents de ceux que je souhaitais utiliser (notamment hors du cadre des LLMs). J'ai donc été, pour une grande partie du travail, seul face aux difficultés techniques rencontrées, avec le soutien ponctuel d'Estelle Zheng lorsque nécessaire.

Ce travail a représenté une part très importante de mon stage, non seulement par le temps passé, mais surtout par le niveau de compréhension que cela a exigé. Il ne s'agissait pas uniquement de faire fonctionner une méthode, mais de la reconstruire en explorant la structure interne des parties des modèles eux-mêmes.

8.2 Comprendre le fonctionnement du Ladder Side Tuning

Un modèle LLM est composé de plusieurs couches identiques d'une structure spécifique de neurones. Cette structure est appelée une couche d'attention. Chaque couche d'attention est séparée par des réseaux de neurones "classiques" appelés MLP (multi layer perceptron). La méthode LST consiste à ajouter à un modèle existant un second plus petit modèle parallèle au réseau principal. Ce réseau secondaire est composé de ses propres couches de neurones récupérées à partir des couches les plus importantes du modèle principal. Les couches de ce nouveau modèle sont liées entre elles, et aussi aux couches du modèle principal, formant les "barreaux" de l'échelle.

Ces liens uni-directionnels du modèle principal vers le modèle secondaire se combinent avec les liens entre les couches du réseau latéral.

Lors de l'adaptation, seul le réseau latéral est modifié et les paramètres du modèle principal restent inchangés. C'est une subtilité qui n'est possible que parce que les couches du réseau latéral sont isolées du modèle principal, contrairement aux autres méthodes d'adaptation qui

sont entre les couches ou à l'intérieur des couches du modèle principal.

Cela permet d'adapter un modèle à une nouvelle tâche sans avoir besoin de reconsidérer ou recalculer une seule de ses composantes. L'efficacité de cette méthode repose en grande partie sur le fait que seuls les éléments nouveaux sont ajustés, ce qui réduit fortement l'usage de mémoire et rend l'adaptation possible sur du matériel plus léger.

J'ai aussi expérimenté une version plus compacte du réseau secondaire, en réduisant progressivement la taille des dimensions des matrices internes des couches du réseau latéral, dans l'idée de garder une représentation cohérente tout en réduisant encore plus les ressources nécessaires. J'ai essayé différentes proportions, sans toujours parvenir à des résultats concluants. Cette phase a été difficile, notamment car elle m'a confronté à des opérations internes au modèle que je ne comprenais pas encore bien, en particulier les calculs liés à la structure même des opérations mathématiques du mécanisme d'attention, qui repose sur des multiplications de matrices non carrées.

8.3 Sélection des parties essentielles du modèle

Avant même de construire le réseau secondaire, il m'a fallu identifier quelles parties du modèle principal étaient les plus utiles à conserver. Cela m'a conduit à utiliser une méthode appelée information de Fisher [7], suggérée comme étant celle qui offrait les meilleurs résultats par le papier, qui mesure l'importance relative de différentes étapes du modèle dans sa capacité à traiter l'information.

J'ai découvert qu'en ne gardant que 4 couches du modèle (sur les 24 que compte le modèle original), et en les combinant à une adaptation complémentaire (fine-tuning), il était possible de maintenir un niveau de performance étonnamment élevé. Par exemple, sur la tâche de compréhension de texte MNLI étudiée précédemment, j'ai obtenu un taux d'exactitude de 34,585% (**Figure 10.4**), ce qui est équivalent à celui du modèle complet non entraîné dans cette configuration de test. Cette observation est illustrée dans la **Figure 8.1**, où l'on voit clairement que seules deux couches du modèle sont porteuses d'une importance très supérieure aux autres (échelle logarithmique).

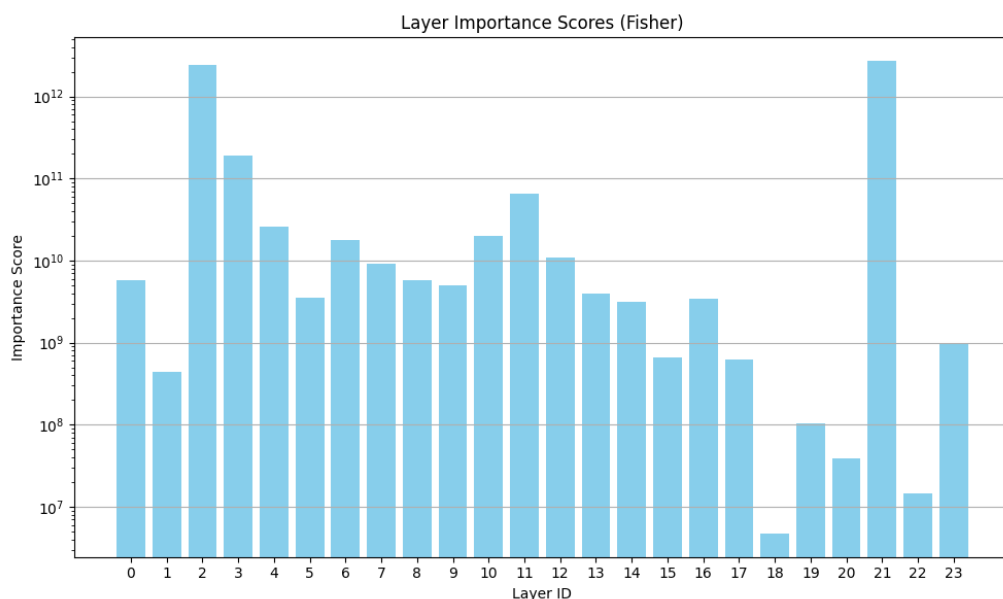


FIGURE 8.1 – Importance relative des différentes couches du modèle selon la méthode d'information de Fisher (échelle logarithmique).

Ce résultat surprenant met en avant la redondance dans les grands modèles actuels. Les notions d'explicabilité des modèles d'IA font l'objet de l'attention de beaucoup de chercheurs,

y compris dans l'équipe SyNaLP. Néanmoins il est important de noter que cette méthode d'évaluation (l'information de Fisher) semble imparfaite lorsqu'on l'applique à des modèles de langage. En effet, elle met beaucoup en avant les MLPs (réseaux de neurones classiques situés entre chaque couche d'attention), dont les dimensions sont bien plus grandes que celles des couches d'attention. Les couches mises en avant par la méthode ont l'air situées de manière aléatoire dans l'ordre des couches du modèle, et tend à sous considérer les couches d'attention, qui sont pourtant l'avancée technique à la base du fonctionnement des LLMs. Cela suggère que cette méthode n'est peut-être pas adaptée pour guider la création du réseau secondaire dans le contexte d'un LLM. Ce choix de méthode n'a pas vraiment influencé la qualité du modèle final car les gains de l'utilisation de cette méthode présentée par le papier étaient de moins d'un pourcent.

8.4 Résultats obtenus et limites rencontrées

Après avoir construit mon propre modèle avec son réseau secondaire, j'ai repris la méthode d'adaptation décrite plus haut. J'ai testé différentes tailles pour ce réseau, en ajustant son architecture afin de trouver un équilibre entre mémoire utilisée et performances obtenues. Au final, avec un réseau secondaire composé de dix couches non redimensionnées, j'ai atteint un taux d'exactitude de 59,94% (**Figure 10.3**) sur la tâche de test, soit exactement le même que celui que j'avais obtenu avec une méthode d'adaptation plus classique (LoRA) en début de stage.

Je n'ai pas eu le temps de mesurer précisément les économies de mémoire obtenues, mais d'après les résultats rapportés dans l'article scientifique ainsi que les travaux de ma tutrice, elles sont considérables, et autour des 63% de mémoire économisée. Cela signifie que l'on peut obtenir des performances comparables en utilisant beaucoup moins de ressources, il serait donc intéressant d'explorer cette technique plus en profondeur.

8.5 Conclusion

Cette expérience m'a demandé une implication très importante, tant en termes de compréhension que de développement. Elle m'a permis d'entrer dans le détail du fonctionnement interne d'un grand modèle de langue, non plus simplement comme un utilisateur, mais comme quelqu'un capable d'en modifier l'architecture. Il ne s'agissait plus seulement d'exécuter du code, mais de comprendre pourquoi et comment il fonctionne, de redéfinir ses composants et de faire face à des problèmes complexes sans solution publiquement accessible.

En tentant de redimensionner les étapes du modèle, j'ai aussi touché aux limites de mes connaissances, notamment en mathématiques, ce qui m'a montré le degré de sophistication réelle des LLMs. Ce travail représente la moitié de mon stage et a été l'occasion d'acquérir une compréhension beaucoup plus structurelle de ce qu'est un modèle de langue, au-delà de son comportement en surface ou de ses propriétés.

Chapitre 9

Conclusion

9.1 Bilan des acquis

Grâce à mes expériences professionnelles préalables, j’ai pu aborder ce stage avec des bases solides qui m’ont permis d’être rapidement autonome sur les aspects techniques des outils utilisés dans la recherche. Ma maîtrise de l’environnement Linux (Ubuntu), du terminal de commandes, et des connexions à distance via SSH, ou encore la gestion des environnements de développement Python, n’ont pas représenté de difficulté particulière. J’étais aussi déjà familiarisé avec le langage Python, la manipulation de données, et la structuration de projets logiciels grâce à mon année d’expérience professionnelle en tant que développeur back-end, ainsi qu’à ma licence professionnelle en développement web ; même si ce n’est pas le langage que je maîtrisais le plus.

J’avais également déjà découvert les modèles de langues de manière superficielle via l’usage d’APIs (connexion distante informatique à un logiciel) externes d’OpenAI pour des projets personnels et pour le travail. Ces compétences m’ont permis de démarrer avec des pré-acquis sur les spécificités du domaine de l’IA, sans être freiné par des difficultés techniques de trop bas niveau. Cela a grandement facilité mon apprentissage et m’a permis de me plonger directement dans les enjeux concrets de l’adaptation et de l’optimisation des modèles de langues plus que la difficulté technique à manipuler des données.

9.2 Réflexions personnelles sur le stage

Ce stage a représenté une opportunité très importante pour entrer en profondeur dans un domaine qui m’a donné envie de reprendre mes études en L3. J’ai pu concentrer tous mes efforts sur un sujet qui me passionne, dans un cadre à la fois exigeant et stimulant, tout en étant utile à un projet très concret.

Je me rends compte de la complexité et de la richesse de ce sujet spécifique dans le domaine de la recherche en IA. Pour l’instant, je n’en comprends qu’une petite partie, et j’ai l’impression de me trouver face à une véritable montagne de connaissances qui me restent à assimiler.

La progression des tâches qui m’ont été confiées a été très bien pensée et je remercie encore Estelle Zheng pour avoir choisi des tâches à me confier en priorisant l’approfondissement de mes compétences. Chaque nouvelle étape s’est révélée plus complexe que la précédente, me forçant à sortir de ma zone de confort et à monter en compétence pour pouvoir continuer. Cette montée en difficulté m’a permis de passer d’une compréhension assez abstraite de ce que sont les modèles de langues à une vision claire et détaillée de leur fonctionnement et des principes fondamentaux de leur entraînement.

Au-delà du travail technique, ce stage a été très enrichissant humainement. Évoluer dans un laboratoire de recherche comme le LORIA, au contact quotidien de doctorants, de chercheurs, d’ingénieurs et de passionnés du monde entier, a été une expérience particulièrement motivante.

Les échanges, qu'ils soient formels ou informels, les séminaires auxquels j'ai pu assister, les discussions autour des projets de chacun. . . tout cela m'a permis de mieux percevoir la richesse et la diversité des travaux de recherche en IA. De plus entre stagiaires nous nous entre-aidions, et partagions les compétences similaires que nous acquiérions au long du stage, ce qui nous a aidées à aller de l'avant en cas de blocage.

9.3 Ouvertures possibles et continuité du travail

Les méthodes que j'ai explorées semblent très prometteuses et il me semble évident que dans le futur elles occuperont une part plus importante des discussions autour de l'allègement de l'entraînement des modèles de langues d'IA. Mon travail autour de l'adaptation de modèles via la méthode LST notamment, ouvre la porte à des recherches complémentaires sur la réduction de la taille des réseaux latéraux, l'optimisation de leur structure ou encore leur combinaison avec d'autres techniques de compression de modèle.

D'autres pistes restent à creuser autour de la sélection des couches les plus informatives dans les modèles de langues. L'inefficacité relative de certaines méthodes actuelles, comme l'information de Fisher appliquée aux LLMs, suggère qu'il existe encore beaucoup à comprendre sur la distribution réelle de l'information dans ces architectures de neurones. Comme mentionné plus haut j'ai pu assister à une présentation d'un post-doctorant travaillant sur ce sujet depuis 2020, qui a obtenu des résultats très intéressants sur l'explicabilité et l'interprétabilité des modèles.

Sur un plan plus personnel, ce stage a renforcé mon intérêt pour la recherche appliquée et m'a donné envie de continuer à explorer ce domaine. Je repars avec des compétences techniques renforcées, une bien meilleure compréhension de l'IA moderne, et la conviction que c'est un champ dans lequel j'ai envie de continuer m'investir à l'avenir.

Chapitre 10

Annexes

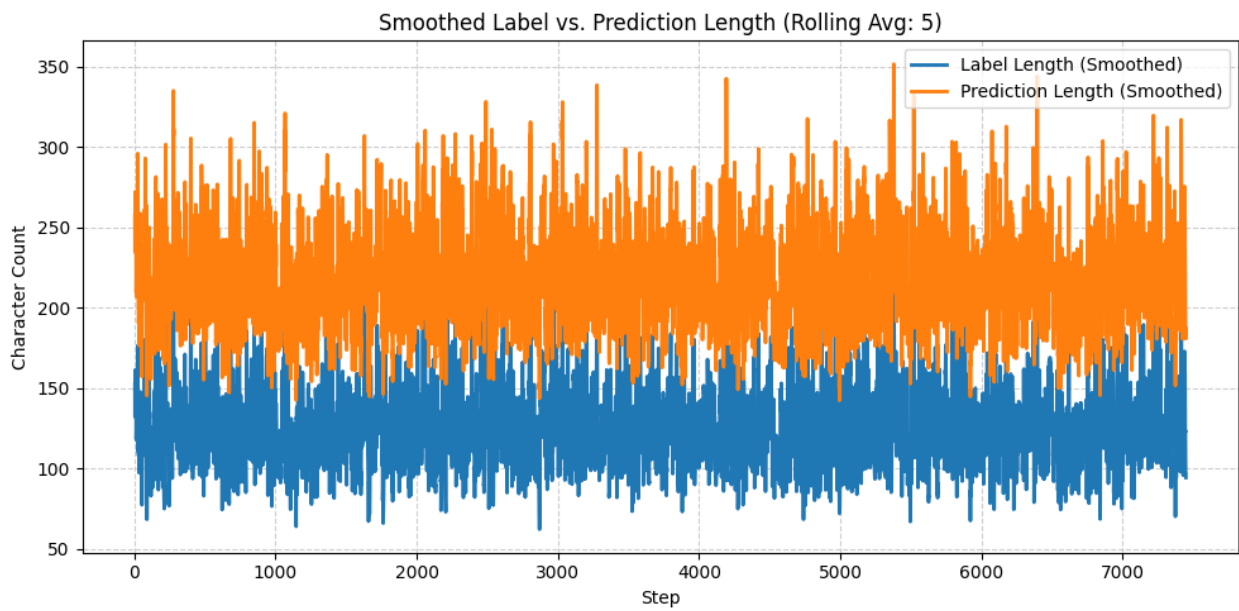
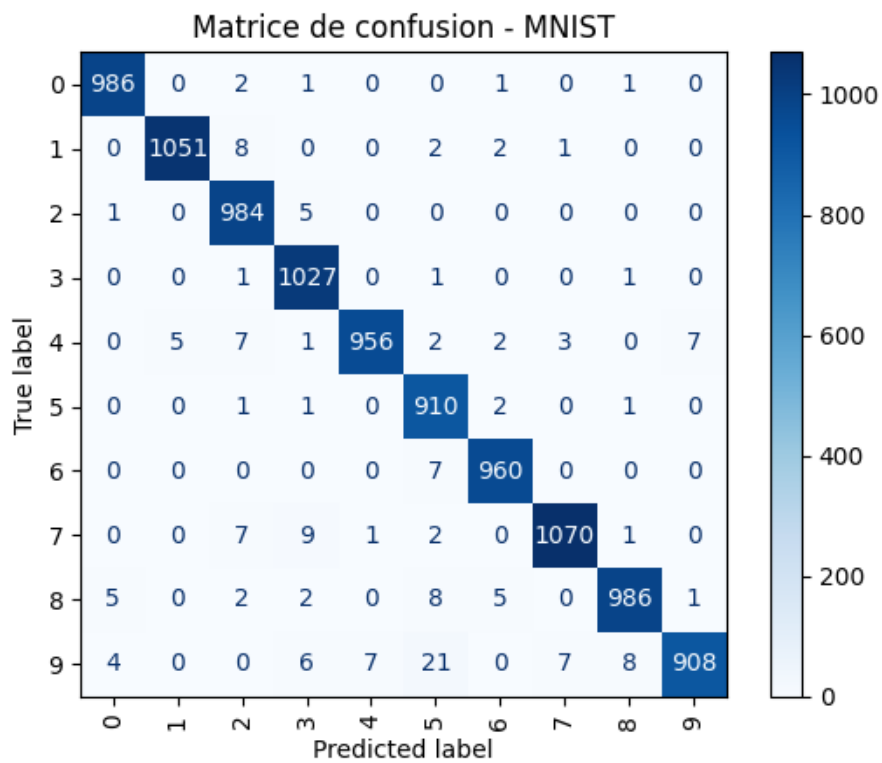


FIGURE 10.1 – Ce graphique présente la longueur de la réflexion du jeu de données (en bleu) comparée à la longueur de la réflexion prédite par le modèle (en orange). La longueur est mesurée en nombre de tokens, et les courbes ont été lissées à l'aide d'une moyenne glissante (rolling average) sur 5 étapes.



Rapport de classification :

	precision	recall	f1-score	support
0	0.99	0.99	0.99	991
1	1.00	0.99	0.99	1064
2	0.97	0.99	0.98	990
3	0.98	1.00	0.99	1030
4	0.99	0.97	0.98	983
5	0.95	0.99	0.97	915
6	0.99	0.99	0.99	967
7	0.99	0.98	0.99	1090
8	0.99	0.98	0.98	1009
9	0.99	0.94	0.97	961
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000

FIGURE 10.2 – Matrice de confusion des résultats du modèle CNN sur le jeu de données MNIST.

Step	Epoch	Loss	Accuracy
450	1	0.5273	53.43%
900	1	0.0334	
1350	1	0.3418	
1800	1	0.2070	
2000	1	0.4043	
2250	1	0.1650	
2700	1	0.0315	
3150	1	0.1680	
3600	1	0.3926	
4000	1	0.5430	
4050	1	0.4336	57.410000000000004%
4500	1	0.1108	
4950	1	0.4941	
5400	1	0.2158	
5850	1	0.1172	
6000	1	0.1846	
6300	1	0.3672	
6750	1	0.4531	
7200	1	0.2334	
7650	1	0.1787	
8000	1	0.5391	59.5%
8100	1	0.0491	
8550	1	0.3730	
9000	1	0.4238	
9450	1	0.0767	
9900	1	0.1768	
10000	1	0.1924	

Evaluation score of the ladder after fine tuning : 59.940000000000005

FIGURE 10.3 – Trace du fine-tuning de la "Ladder" construite à partir du modèle Qwen-2.5-0.5B-Instruct qui présente une exactitude de 59,94%, entraînement sur 10 000 exemples issus du jeu de données "MNLI".

Training Log

Step	Epoch	Loss	Accuracy
600	1	17.7935	
1200	1	17.2145	
1800	1	16.6588	
2400	1	12.5923	
2500	1	11.8476	0.0%
3000	1	11.3071	
3600	1	3.5713	
4200	1	2.1470	
4800	1	1.0897	
5000	1	5.3193	0.3125%
5400	1	4.6875	
6000	1	0.9726	
6600	1	3.5487	
7200	1	2.7586	
7500	1	0.7015	6.4575%
7800	1	0.7570	
8400	1	0.6385	
9000	1	0.6037	
9600	1	0.5738	
10000	1	0.6206	33.410000000000004%
10200	1	0.5764	
10800	1	0.5761	
11400	1	0.7941	
12000	1	0.4339	
12500	1	0.4320	33.93%
12600	1	0.4516	
13200	1	0.3392	
13800	1	0.5061	
14400	1	0.5158	
15000	1	0.4479	33.73%
15600	1	0.4417	
16200	1	0.4060	
16800	1	0.4118	
17400	1	0.3500	
17500	1	0.3391	33.777499999999996%
18000	1	0.3933	
18600	1	0.4201	
19200	1	0.4707	
19800	1	0.3652	
20000	1	0.4385	34.57%
20400	1	0.4948	
21000	1	0.3692	
21600	1	0.4498	

34.585
(unslloth-env) achoplin@gres-5:~/lst\$

FIGURE 10.4 – Trace du fine-tuning QLoRa du modèle Qwen-2.5-0.5B-Instruct pruné, dont seules quatres couches restent qui présente une exactitude de 34,585%, entraînement sur 22 000 exemples issus du jeu de données "MNLI".

Bibliographie

- [1] Inc. ANACONDA. *conda Package — Anaconda Cloud*. Accessed : 2025-06-02. 2025. URL : <https://anaconda.org/anaconda/conda>.
- [2] CLOUDFLARE. *What is a Large Language Model (LLM) ?* Consulté le 15 mai 2025. 2024. URL : <https://www.cloudflare.com/fr-fr/learning/ai/what-is-large-language-model/>.
- [3] Raluca CSERNATONI. *The EU's AI Power Play : Between Deregulation and Innovation*. Accessed : 2025-06-02. Carnegie Europe, mai 2025. URL : <https://carnegieendowment.org/research/2025/05/the-eus-ai-power-play-between-deregulation-and-innovation?lang=en>.
- [4] Tim DETTMERS et al. *QLoRA : Efficient Finetuning of Quantized LLMs*. 2023. arXiv : 2305.14314 [cs.LG]. URL : <https://arxiv.org/abs/2305.14314>.
- [5] Digital EUROPE. *Digital Europe*. 1999. URL : <https://www.digitaleurope.org/>.
- [6] Horizon EUROPE. *Horizon Europe*. 2021. URL : https://commission.europa.eu/funding-tenders/find-funding/eu-funding-programmes/horizon-europe_en.
- [7] *Fisher Information*. URL : https://fr.wikipedia.org/wiki/Information_de_Fisher.
- [8] Leo GAO et al. *The Language Model Evaluation Harness*. Version v0.4.3. Juill. 2024. URL : <https://zenodo.org/records/12608602>.
- [9] Thibaud MICHARD. *Le Fine-Tuning de LLM devient obsolète ?* Accessed : 2025-06-02. Reglo.ai, juin 2024. URL : <https://reglo.ai/fine-tuning-llm/>.
- [10] MOSAIK TEAM. *Welcome to the MosAIk team*. Accessed : 2025-06-02. Loria, CNRS, CentraleSupélec, University of Lorraine, 2024. URL : <https://site-23068d.gitlabpages.inria.fr/>.
- [11] Yi-Lin SUNG, Jaemin CHO et Mohit BANSAL. *LST : Ladder Side-Tuning for Parameter and Memory Efficient Transfer Learning*. NeurIPS 2022, version 2, last revised 31 Oct 2022. 2022. URL : <https://arxiv.org/abs/2206.06522>.
- [12] Qwen TEAM. *Qwen2.5 : A Party of Foundation Models*. Sept. 2024. URL : <https://qwenlm.github.io/blog/qwen2.5/>.

Glossaire

- API** Application Programming Interface, ensemble de définitions et protocoles permettant à des logiciels de communiquer entre eux. 22
- CentraleSupélec** Grande école d'ingénieurs française spécialisée en sciences de l'ingénieur, membre de l'Université Paris-Saclay. 7
- Chain of Thought (CoT)** Technique consistant à générer un raisonnement intermédiaire explicite pour améliorer la exactitude des modèles sur les tâches complexes. 17
- ChatGPT** Modèle de langage développé par OpenAI, conçu pour générer du texte conversationnel. 4, 5
- CIFRE** Convention Industrielle de Formation par la Recherche, dispositif permettant à un doctorant d'effectuer sa thèse en collaboration avec une entreprise. 7
- CNN** Convolutional Neural Network, ou réseau de neurones convolutif, utilisé principalement pour le traitement d'images. 16, 17, 25
- CNRS** Centre National de la Recherche Scientifique, principal organisme public de recherche en France. 6–8
- Conda** Gestionnaire d'environnements et de paquets pour Python, utilisé pour isoler des configurations logicielles. 15
- CUDA** Plateforme de calcul parallèle développée par NVIDIA pour exploiter la puissance des cartes graphiques. 15
- drivers** Logiciels permettant au système d'exploitation de communiquer avec le matériel, comme une carte graphique. 15
- fine-tuning** Technique d'adaptation d'un modèle de langage pré-entraîné à une tâche spécifique, en le réentraînant sur un jeu de données ciblé. 8, 14–16, 20
- GAFAM** Acronyme désignant les cinq géants américains du numérique : Google, Apple, Facebook (Meta), Amazon et Microsoft. 5
- GitHub** Plateforme en ligne d'hébergement de code et de collaboration basée sur le système de versionnage Git. 17
- Grid5000** Plateforme de calcul distribuée française dédiée à la recherche expérimentale en informatique, composée de serveurs répartis dans plusieurs villes. 8, 10, 15
- GSM8K** Jeu de données de 8 500 problèmes mathématiques de niveau scolaire, souvent utilisé pour évaluer les capacités de raisonnement numérique des modèles de langue. 17
- Huggingface** Plateforme et bibliothèque open source dédiée aux modèles de langage et à leur entraînement. 10, 11, 16, 17
- hyperparamètres** Paramètres définis avant l'entraînement d'un modèle et qui influencent son comportement (ex. taux d'apprentissage, batch size). 14, 17

IA Intelligence Artificielle : ensemble de techniques permettant à des machines d’accomplir des tâches habituellement réalisées par l’humain. 4, 19, 20, 22, 23

information de Fisher Mesure statistique utilisée pour estimer l’importance des paramètres d’un modèle dans sa capacité à traiter l’information. 23

Inria Institut national de recherche en sciences et technologies du numérique, établissement public français dédié à la recherche en informatique. 7

intelligence artificielle Ensemble de méthodes permettant à des machines d’accomplir des tâches nécessitant normalement une intelligence humaine. 14, 16

jeu de données Ensemble structuré d’exemples utilisés pour entraîner ou évaluer un modèle d’apprentissage automatique. 16, 17

LLM Large Language Model (Grand Modèle de Langue) : modèle d’intelligence artificielle entraîné sur de grandes quantités de texte pour accomplir des tâches comme la génération, la traduction ou le résumé de texte. 5, 7, 8, 10, 17–19, 21, 23

lm-eval-harness Outil d’évaluation standardisé pour comparer les performances des modèles de langage sur des tâches précises. 13, 16, 17

LoRA Low-Rank Adaptation, méthode PEFT consistant à ajouter des modules légers au modèle sans toucher à sa structure initiale. 11, 14, 21

LORIA Laboratoire Lorrain de Recherche en Informatique et ses Applications, centre de recherche en informatique basé à Nancy. 22

LST Ladder Side Tuning, méthode d’adaptation de modèle en ajoutant une structure parallèle. 14, 19, 23

matrice de confusion Tableau permettant de visualiser les performances d’un modèle de classification, en comparant les prédictions aux vraies classes. 17

Mistral Famille de modèles de langage développée en Europe, réputée pour sa performance et son efficacité. 4

MLP Multi Layer Perceptron : réseau de neurones artificiel constitué de plusieurs couches entièrement connectées, utilisé comme composant classique dans les architectures de réseaux neuronaux. 19, 21

MNIST Jeu de données contenant des images de chiffres manuscrits (0 à 9), couramment utilisé pour entraîner des modèles de vision par ordinateur. 16, 25

MNLI Multi-Genre Natural Language Inference, jeu de données d’inférence logique entre phrases. 20

modèle de langue Modèle d’intelligence artificielle capable de comprendre et de générer du langage naturel. 15, 16, 18, 21–23

modèle fondation Modèle d’IA de très grande taille, préentraîné sur des données massives, servant de base pour de nombreuses tâches ou spécialisations. 5

neurones Unités de calcul de base dans un réseau de neurones artificiels. 15, 16, 19, 21, 23

optimiseur Algorithme utilisé pour ajuster les paramètres d’un modèle de manière à minimiser l’erreur, en se basant sur les gradients calculés. 17

PEFT Parameter-Efficient Fine-Tuning, ensemble de techniques visant à adapter un modèle en ne modifiant qu’un petit nombre de paramètres. 11

- Python** Langage de programmation largement utilisé pour le développement et l’entraînement de modèles d’intelligence artificielle. 4, 14–16, 22
- PyTorch** Bibliothèque open-source de machine learning basée sur Torch, utilisée pour créer et entraîner des réseaux de neurones. 16, 17
- réseau latéral** Second réseau plus petit ajouté parallèlement au réseau principal dans la méthode Ladder Side Tuning, permettant l’adaptation sans modifier le modèle principal. 23
- SSH** Secure Shell, protocole de communication sécurisée permettant d’accéder à distance à un terminal informatique. 22
- SyNaLP** Équipe de recherche du LORIA spécialisée en syntaxe, sémantique et traitement automatique du langage naturel. 21
- tokens** Unités de texte (souvent des mots, sous-mots ou caractères) utilisées en entrée et sortie par les modèles de langage. 17, 18, 24